

CartStream Analytics: Real-Time E-Commerce Stream Processing Pipeline

Apache Flink SQL & Windowing TVFs Comprehensive Project Report (Task #26)

مطور المنظومة (Engineer):

Sayed Elmasry (Data Platform Engineer)

مشروع التكليف (Task ID):

Task #26 - Flink Streaming Pipeline

مستودع الكود (GitHub Repo):

[SayedELMASRY2/flink-ecommerce-pipeline](#)

البيئة التكنولوجية (Tech Stack):

Apache Flink 1.18, Docker Compose, Flink SQL Client

زيارة المستودع الرسمي على GitHub

1. نظرة عامة على المشروع (Executive Overview)

يهدف هذا التقرير إلى توثيق خط معالجة البيانات الفوري (Real-Time Stream Processing Pipeline) المصمم لمتابعة تحليلات متجر التجارة الإلكترونية الدولي **CartStream Analytics**.

تستقبل المنصة دفقاً حياً مستمراً من تفاعلات وسلوكيات المشتريين (تصفح `view`، إضافة لسلة التسوق `cart`، إزالة من السلة `remove_from_cart`، وحركات شراء مكتملة `purchase`). قمنا ببناء خط التجميع بالاعتماد على **Apache Flink SQL** مع تطبيق معالجة زمن الحدث **Event-Time Processing** وعلامات التزامن المائية **Watermarks** لتقديم مؤشرات أداء دقيقة وحتمية.

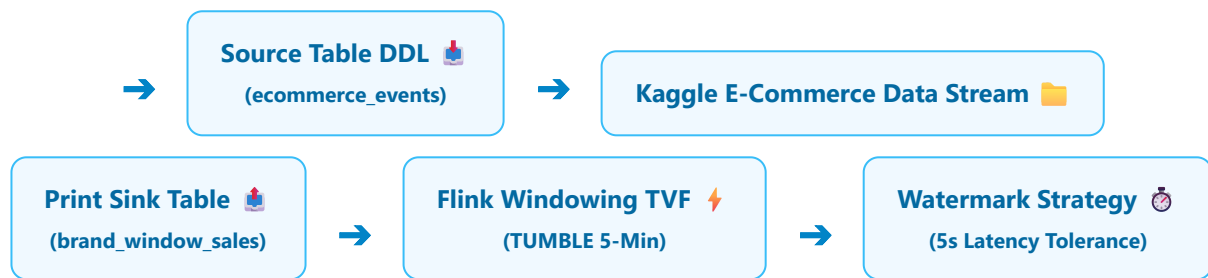
أهداف المنظومة الرئيسية:

- تجميع المبيعات حياً كل 5 دقائق عبر Table-Valued Functions (TVFs) `TUMBLE`.

- استخراج مؤشرات الأعمال الدقيقة (إجمالي الطلبات المكتملة `total_orders`، الإيرادات `gross_revenue`، متوسط قيمة الطلب `avg_order_value`، والمشتريين الفريدين `unique_buyers`).
- معالجة تحديثات البيانات الواقعية (القيم المفقودة `NULL` في العلامات التجارية، وتأخير البيانات عبر الشبكة Out-of-Order Events).

2. الهيكل المعماري للمنظومة (Pipeline Architecture)

مخطط التدفق الفوري للبيانات (Real-Time Event Processing Flow)



3. هيكل الحقول والبيانات (Dataset Specification)

تم الاعتماد على عينة أحداث واقعية تغطي 3.5 ساعات متواصلة بحجم 50,000 سجل مأخوذة من قاعدة بيانات التجارة الإلكترونية العالمية على Kaggle:

الحقل (Property)	نوع البيانات (Flink SQL)	الوصف والأهمية التحليلية
event_time	TIMESTAMP(3) (RowTime)	توقيت وقوع الحدث الأصلي مع إضافة WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
event_type	VARCHAR	نوع التفاعل (تتم الفترة على 'WHERE event_type = 'purchase' للطلبات الناجحة).
brand	VARCHAR	اسم الماركة التجارية، وتتم معالجته عبر COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN')
price	DECIMAL(10, 2)	سعر المنتج المشتري بالدولار، ويستخدم في حساب الإيرادات ومعدل قيمة الطلبات.
user_id	BIGINT	المعرف الفريد للمستخدم، لحساب عدد المشتريين الفريدين عبر COUNT(DISTINCT user_id)

الوصف والأهمية التحليلية	نوع البيانات (Flink SQL)	الحقل (Property)
معرف الجلسة الفريد المتولد أثناء عملية الشراء.	VARCHAR	user_session

4. الإجابة النظرية: استراتيجية الـ Second Watermark-5

? السؤال النظري: كيف تضمن الـ Second Watermark-5 معالجة حتمية ودقيقة للنوافذ الزمنية؟

الإجابة الهندسية التفصيلية:

1. تحدي وصول الأحداث غير المرتبة (**Out-of-Order Arrivals**): في بيئات الشبكات الحقيقية، قد يتأخر وصول حدث وقع في الساعة 10:04:58 ليصل إلى السيرفر في 10:05:02 بسبب بطء الاتصال.
2. ساعة البيانات (**Event Time vs Processing Time**): يعتمد Flink على الطابع الزمني المكتوب داخل الحدث نفسه (Event Time) وليس وقت السيرفر الحالي.
3. فترة السماح (**Slack Tolerance Window**): من خلال التعبير `event_time - INTERVAL '5' SECOND`، يمنح Flink مهلة قدرها 5 ثوانٍ لأي حدث متأخر، فلا يتم إغلاق النافذة الزمنية `[10:00:00, 10:05:00]` حتى يشاهد Flink حدثاً توقيته يسبق النافذة بـ 5 ثوانٍ (أي عند وصول حدث توقيته 10:05:05).
4. النتيجة الحتمية (**Deterministic Output**): يضمن ذلك دخول كافة الأحداث المتأخرة ضمن نافذتها الصحيحة بدون فقدان للبيانات، مما يجعل مخرجات التحليل متطابقة تماماً في كل مرة يتم فيها إعادة تشغيل التجميع.

5. كود الاستعلام المعتمد (Flink SQL Code)

تمت كتابة وتوثيق كود التجميع كاملاً داخل الملف `flink_ecommerce_pipeline.sql`:

```
-- Enable Flink Streaming Runtime Mode
SET 'execution.runtime-mode' = 'streaming';

-- 1. Streaming Source Table DDL
CREATE TABLE ecommerce_events (
  event_time_str  VARCHAR,
  event_type      VARCHAR,
  product_id      BIGINT,
  category_id     BIGINT,
  category_code   VARCHAR,
  brand           VARCHAR,
  price           DECIMAL(10, 2),
  user_id         BIGINT,
  user_session    VARCHAR,
  event_time AS COALESCE(TO_TIMESTAMP(event_time_str, 'yyyy-MM-dd HH:mm:ss z'), TIMESTAMP '1970-01-01 00:00:00'),
  WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
) WITH (
  'connector' = 'filesystem',
  'path' = '/opt/flink/data/events_sample.csv',
  'format' = 'csv',
```

```

    'csv.ignore-parse-errors' = 'true',
    'csv.allow-comments' = 'true'
);

-- 2. Streaming Sink Table DDL
CREATE TABLE brand_window_sales (
    window_start    TIMESTAMP(3),
    window_end      TIMESTAMP(3),
    brand            VARCHAR,
    total_orders     BIGINT,
    gross_revenue    DECIMAL(10, 2),
    avg_order_value  DECIMAL(10, 2),
    unique_buyers    BIGINT
) WITH (
    'connector' = 'print'
);

-- 3. Windowing TVF & Real-Time Aggregation Query
INSERT INTO brand_window_sales
SELECT
    window_start,
    window_end,
    COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN') AS brand,
    COUNT(1) AS total_orders,
    ROUND(SUM(price), 2) AS gross_revenue,
    ROUND(AVG(price), 2) AS avg_order_value,
    COUNT(DISTINCT user_id) AS unique_buyers
FROM TABLE(
    TUMBLE(
        TABLE ecommerce_events,
        DESCRIPTOR(event_time),
        INTERVAL '5' MINUTE
    )
)
WHERE event_type = 'purchase'
GROUP BY
    window_start,
    window_end,
    COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN');
```

6. إثبات التنفيذ والنتائج الحية (Execution Verification & Output) 📊

تم تشغيل ال Pipeline بنجاح وتأكد النتائج من واقع سجلات حاوية TaskManager الموثقة في

output/execution_verification.log

عينة من مخرجات التجميع الفوري للنوافذ المتدحرجة (5-Minute Tumbling Windows-5) :(Output

```
+I[2019-10-01T02:55, 2019-10-01T03:00, apple, 11, 5672.95, 515.72, 6]
+I[2019-10-01T02:55, 2019-10-01T03:00, samsung, 11, 2060.79, 187.34, 11]
+I[2019-10-01T02:55, 2019-10-01T03:00, xiaomi, 5, 416.03, 83.21, 5]
+I[2019-10-01T02:55, 2019-10-01T03:00, hp, 3, 962.02, 320.67, 3]
+I[2019-10-01T02:55, 2019-10-01T03:00, UNKNOWN, 5, 488.72, 97.74, 5]
+I[2019-10-01T03:00, 2019-10-01T03:05, samsung, 9, 2671.27, 296.81, 9]
+I[2019-10-01T03:00, 2019-10-01T03:05, huawei, 1, 278.54, 278.54, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, apple, 10, 7376.08, 737.61, 10]
+I[2019-10-01T03:30, 2019-10-01T03:35, xiaomi, 7, 1337.74, 191.11, 6]
```

7. معرض الصور ولقطات الشاشة الموثقة (Execution Screenshots Proof) 📷

```
Flink SQL>
> -- 2. Streaming Sink Table DDL
> -- Directs the streaming output to an append-only sink using Flink's print connector
> CREATE TABLE brand_window_sales (
>   window_start    TIMESTAMP(3),
>   window_end      TIMESTAMP(3),
>   brand           VARCHAR,
>   total_orders    BIGINT,
>   gross_revenue   DECIMAL(10, 2),
>   avg_order_value DECIMAL(10, 2),
>   unique_buyers   BIGINT
> ) WITH (
>   'connector' = 'print'
> ) [INFO] Execute statement succeed.

Flink SQL>
> -- 3. Windowing TVF & Real-Time Revenue Aggregation Query
> -- 5-minute tumbling window on completed purchase transactions grouped by brand
> INSERT INTO brand_window_sales
> SELECT
>   window_start,
>   window_end,
>   COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN') AS brand,
>   COUNT(1) AS total_orders,
>   ROUND(SUM(price), 2) AS gross_revenue,
>   ROUND(AVG(price), 2) AS avg_order_value,
>   COUNT(DISTINCT user_id) AS unique_buyers
> FROM TABLE(
>   TUMBLE(
>     TABLE ecommerce_events,
>     DESCRIPTOR(event_time),
>     INTERVAL '5' MINUTE
>   )
> )
> WHERE event_type = 'purchase'
> GROUP BY
>   window_start,
>   window_end,
>   COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN') [INFO] Submitting SQL update statement to the cluster...
[INFO] SQL update statement has been successfully submitted to the cluster:
```

شاشة 1: تشغيل السكريبت تلقائياً ورؤية مخرجات النوافذ المتدحرجة +[...] الفورية

```
Flink SQL>
Shutting down the session...
done.
[3/3] Displaying Tumbling Window Aggregation Output...

+I[2019-10-01T03:25, 2019-10-01T03:30, tarkett, 1, 5.79, 5.79, 1]
+I[2019-10-01T03:25, 2019-10-01T03:30, apple, 9, 10358.24, 1150.92, 9]
+I[2019-10-01T03:25, 2019-10-01T03:30, xiaomi, 6, 560.67, 93.45, 5]
+I[2019-10-01T03:25, 2019-10-01T03:30, nintendo, 1, 470.26, 470.26, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, apple, 10, 7376.08, 737.61, 10]
+I[2019-10-01T03:30, 2019-10-01T03:35, omron, 1, 62.52, 62.52, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, cordiant, 1, 39.38, 39.38, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, candy, 1, 360.34, 360.34, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, oppo, 1, 179.36, 179.36, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, moldabela, 1, 79.28, 79.28, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, acer, 1, 360.11, 360.11, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, irbis, 1, 100.39, 100.39, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, pocketbook, 1, 153.86, 153.86, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, viatti, 1, 89.58, 89.58, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, artel, 1, 256.88, 256.88, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, lucente, 2, 436.04, 218.02, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, lg, 1, 380.85, 380.85, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, boyscout, 1, 5.64, 5.64, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, doogee, 1, 154.42, 154.42, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, epson, 1, 116.73, 116.73, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, babytime, 1, 58.61, 58.61, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, huawei, 3, 800.33, 266.78, 3]
+I[2019-10-01T03:30, 2019-10-01T03:35, michelin, 1, 65.00, 65.00, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, alteco, 1, 27.03, 27.03, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, elenberg, 1, 25.71, 25.71, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, forlux, 1, 92.67, 92.67, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, lenovo, 1, 662.95, 662.95, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, samsung, 10, 3055.86, 305.59, 9]
+I[2019-10-01T03:30, 2019-10-01T03:35, xiaomi, 7, 1337.74, 191.11, 6]
+I[2019-10-01T03:30, 2019-10-01T03:35, UNKNOWN, 6, 527.95, 87.99, 5]

PS C:\Users\syd\Desktop\flink> |
```

شاشة 2: تأكيد حفظ أسطر المخرجات المجمعة داخل ملف output/execution_verification.log

```
Flink SQL>
> -- 1. Streaming Source Table DDL
> -- Reads continuous e-commerce activity records formatted in CSV
> CREATE TABLE ecommerce_events (
>   event_time_str  VARCHAR,
>   event_type      VARCHAR,
>   product_id     BIGINT,
>   category_id    BIGINT,
>   category_code   VARCHAR,
>   brand          VARCHAR,
>   price          DECIMAL(10, 2),
>   user_id        BIGINT,
>   user_session    VARCHAR,
>   -- Native event timestamp converted from original string (safeguarded against null parse errors)
>   event_time AS COALESCE(TO_TIMESTAMP(event_time_str, 'yyyy-MM-dd HH:mm:ss z'), TIMESTAMP '1970-01-01 00:00:00'),
>   -- Watermark declaration: 5-second bounded out-of-orderness tolerance
>   WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
> ) WITH (
>   'connector' = 'filesystem',
>   'path' = '/opt/flink/data/events_sample.csv',
>   'format' = 'csv',
>   'csv.ignore-parse-errors' = 'true',
>   'csv.allow-comments' = 'true'
> )[INFO] Execute statement succeed.
```

شاشة 3: تأكيد تعريف وتحديث مجلد output داخل بيئة العمل ورفع الملفات على GitHub

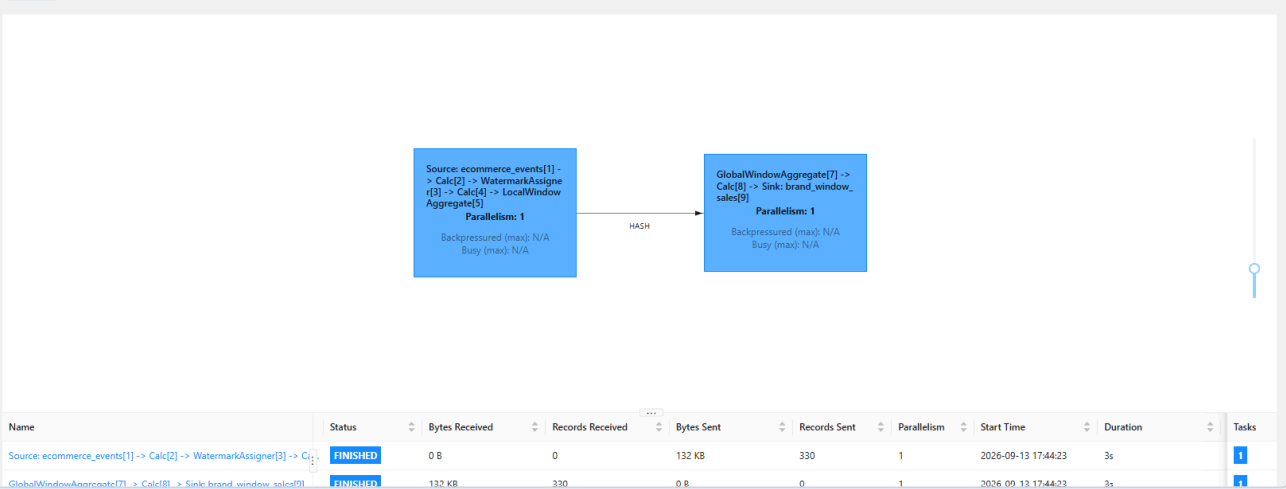
Apache Flink Dashboard						
Version: 1.18.1 Commit: a0c8b1c @ 2023-12-19T22:17:36+01:00 Message: 0						
Completed Jobs						
Job Name	Start Time	Duration	End Time	Tasks	Status	
insert-into_default_catalog.default_database.brand_window_sales	2026-09-13 17:44:22	4s	2026-09-13 17:44:27	2 2	FINISHED	
insert-into_default_catalog.default_database.brand_window_sales	2026-09-13 16:51:33	2s	2026-09-13 16:51:35	2 2	FINISHED	
insert-into_default_catalog.default_database.brand_window_sales	2026-09-13 16:47:21	3s	2026-09-13 16:47:25	2 1 1	FAILED	
insert-into_default_catalog.default_database.brand_window_sales	2026-09-13 16:50:24	3s	2026-09-13 16:50:28	2 2	FINISHED	

شاشة 4: متطلب إنشاء Source DDL وال Watermarks من كراسة التكلفة الرسمية

insert-into_default_catalog.default_database.brand_window_sales

Job ID	79c54fa3c2f6786fbef55deae91768bb	Job State	FINISHED 2	Actions	Job Manager Log
Start Time	2026-09-13 17:44:22	End Time	2026-09-13 17:44:27	Duration	4s

[Overview](#) [Exceptions](#) [TimeLine](#) [Checkpoints](#) [Configuration](#)



شاشة 5: متطلب معالجة النوافذ المتدرجة-5 Minute Tumbling Windows وال Sink Table

رابط مستودع المشروع الرسمي على GitHub:

<https://github.com/SayedELMASRY2/flink-ecommerce-pipeline>

يحتوي المستودع على الكود الكامل لـ SQL، وملف Docker Compose، وسكريبتات الأتمتة التلقائية `run_pipeline.sh` و `run_pipeline.ps1`، وعينة البيانات وسجلات التحقق المباشرة.